How to Build the Course Recommender — Step by Step

This is your **construction manual**. The Handbook told you *what* the project is and *why*. This document tells you *exactly how to build it* — module by module, what goes in, what comes out, and how each piece connects to the next inside one Colab notebook.

`9 modules · 1 notebook` `read top → bottom` `companion to the Handbook`

00

How to use this document

You are building **one Jupyter notebook in Google Colab**. Think of it as a factory line: data enters at the top, and each station (a **module**) does one job and hands its result to the next station. A module is just **one labelled section of the notebook** — usually 1–4 code cells.

> **The golden rule of this project:** the *output* of one module is the *input* of the next. The connection between them is always a **named Python variable**. Build the modules **in order** and keep the variable names exactly as written here, and each step will feed cleanly into the next.

How to work through it

1. Read the Pipeline diagram once, fully. That is the whole project on one screen.
2. Keep the Data Dictionary open in a second tab — it is the master list of every variable that travels between modules. It is your contract.
3. Build **in order, 0 → 7**. Do not start Module 4 before Module 2 produces clean data. Module 8 is optional and last.
4. For your assigned module: read its *Purpose*, copy the *Starter code* into a cell, run it, then check the *"Done when…"* box at the bottom before moving on.
5. When something breaks, check *Common errors* first — your bug is almost certainly listed there.

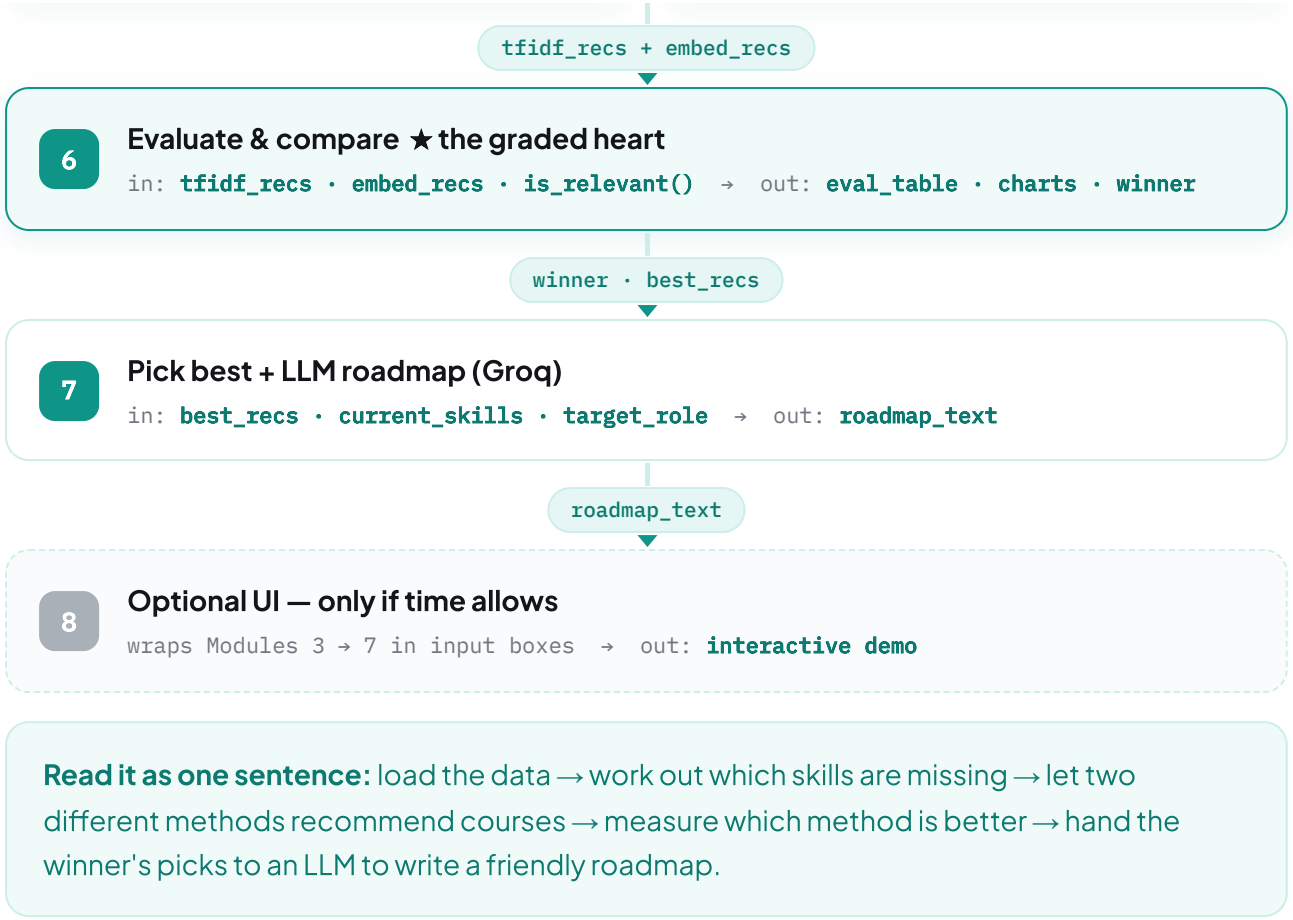
The starter code is a scaffold, not the finished answer. It runs, but it is intentionally minimal. Your job — and the part that earns marks — is to understand it, improve it, and explain it in the report. Copying without understanding will show immediately when you present.

One example runs through this entire document. Meet **Riya**: she already knows `Python`, and she wants to become a `Data Scientist`. Watch her data flow through every module so you can see exactly what each step produces.

01

The full pipeline — how everything connects

Every box is a module (a section of the notebook). Every **labelled arrow is the variable** that one module hands to the next. Modules 4 and 5 are **two different methods**: they both read the same two inputs and each produces its own ranked list, which meet again at the evaluation step (M6), where you compare them.



Read it as one sentence: load the data → work out which skills are missing → let two different methods recommend courses → measure which method is better → hand the winner's picks to an LLM to write a friendly roadmap.

02

The data dictionary — your contract

These are **every variable that travels between modules**. Spell them exactly like this in your code — that consistency is what lets each module's output feed cleanly into the next as you build the notebook in order.

VARIABLE	TYPE	CREATED IN	WHAT IT HOLDS (EXAMPLE)
`courses_raw`	DataFrame	M1	The CSV exactly as downloaded, unmodified
`role_skills`	dict	M1	`{"Data Scientist": ["python","statistics","machine learning", ...]}`
`courses`	DataFrame	M2	Clean table. Required columns: `course_id, title, skills, text` (extra columns like `description, provider` may ride along but aren't used)
`current_skills`	list[str]	M3	`["python"]`
`target_role`	str	M3	`"Data Scientist"` — **must** be a key in `role_skills`
`skill_gap`	list[str]	M3	`["statistics","machine learning","sql","data visualization","deep learning"]`
`query_text`	str	M3	`"statistics machine learning sql data visualization deep learning"`
`tfidf_recs`	DataFrame	M4	Ranked list. Columns: `course_id, title, score, rank`
`embed_recs`	DataFrame	M5	Same shape as `tfidf_recs`
`is_relevant()`	function	M6	Decides if a course counts as a correct recommendation (see §03)
`eval_table`	DataFrame	M6	precision@k for each method, side by side
`winner`	str	M6	`"Embedding"` or `"TF-IDF"`
`best_recs`	DataFrame	M7	The winning method's ranked list (= `tfidf_recs` or `embed_recs`)
`roadmap_text`	str	M7	The month-by-month roadmap the LLM wrote

Build in order. The evaluation (M6) cannot run until both recommenders (M4, M5) produce their `_recs` tables in *exactly* this shape. Agree the column names early and never change them silently.

03

Two rules that are locked — do not redesign these

These two decisions are made *for* you, because getting them wrong quietly breaks everything downstream. Everything else is yours to design.

Rule 1 — What makes a course "relevant" (the ground truth)

To *measure* a recommender, you need a definition of a correct answer. The raw data does not contain one, so we define it:

A recommended course is *relevant* if one of the gap skills appears (as a whole word) in its skills.
i.e. the course teaches at least one skill the person is actually missing.

```
def is_relevant(course_skills, skill_gap):
    # True if a gap skill appears as a whole word in the course's skills
    skills_text = " ".join(course_skills).lower()
    return any(re.search(r"\b" + re.escape(g) + r"\b", skills_text) for g in skill_
```

Why not the simpler `set(course_skills) & set(skill_gap)`? Because the real dataset's skills are *phrases* — e.g. `"Statistics for Data Science"`, not the bare token `"statistics"`. An exact set-match would score `statistics` as **0** relevant courses even though dozens teach it (verified on the real data). Whole-word matching (`\b…\b`) catches `statistics` inside `"statistics for data science"`, yet won't wrongly match `sql` inside `"mysql"`.

This is the single most important rule in the whole project — it is the yardstick every score is measured against. **Discuss its limitations in your report** (e.g. it treats all gap-skills as equally important, and whole-word matching can still miss synonyms like "ML" for "machine learning"). That discussion is worth marks.

Rule 2 — `target_role` must match a known role

The role the user picks must be a key that exists in `role_skills`. If you allow free typing, `"data scientist"` (lowercase) or `"ML Engineer"` will silently find nothing and the pipeline produces empty results with no error.

So: the user *chooses from a fixed list of roles* (the keys of `role_skills`) — never types a role freely. In the optional UI (M8) this becomes a dropdown, not a text box.

04

Weekly plan — what to build, test & demo each week

This is your **sprint plan**. The weeks between Thursday meetings are your sprints. Each sprint you **build** a set of modules, **test-run** them on real data, and **demo them working** at the next meeting. Build strictly in order — a later module can't run until the earlier one produces its output.

A module is "done" only when you have (1) run it on real data, (2) ticked its green *"Done when…"* box, and (3) can *show it working* at the next meeting. Code that "should work" but was never run does not count.

SPRINT	BY THIS MEETING	BUILD	TEST-RUN / DEMO AT THE MEETING
1	Mtg 2 · 25 Jun	M0 Setup · M1 Load data · a *crude* M4 (TF-IDF returns *something*)	Notebook opens in Colab · data loads · `role_skills` works · a rough recommendation prints
2	Mtg 3 · 2 Jul	M2 Clean · M3 Skill gap · finish M4 (clean TF-IDF) · M5 Embeddings	Both recommenders return a top-K list for the same test user
3	Mtg 4 · 9 Jul	M6 Evaluate & compare (precision@k + charts)	Comparison table + bar chart shown · you can say which method won and why
4	Mtg 5 · 16 Jul	M7 LLM roadmap · **feature freeze** · decide on optional M8	Full end-to-end run: input → recommendations → roadmap
5	Mtg 6 · 23 Jul	(optional M8) · clean the notebook · draft your report sections	Notebook runs top-to-bottom with no errors · each owner's draft section
6	Mtg 7 · 30 Jul	Finalise report · dry-run the walkthrough	Full assembled report draft · practice presentation
—	**31 Jul (Fri)**	—	**Submit the final report to ISI**

Timeline reality: kickoff (Fri 19 Jun) to deadline (31 Jul) is ≈ 6 working weeks. It's tight — that's why scope is lean and why you build a *crude* version first (Sprint 1) and improve it, rather than waiting for perfect. **No new features after the Sprint-4 freeze.**

05

The modules, in build order

Build these in sequence. Each one: *what it does → exact inputs → exact outputs → how to build it → Riya's worked example → common errors → done-when.*

MODULE 0 **Setup & environment** 🗓 Sprint 1 · Mtg 2

Purpose. Install the libraries and load your free Groq key, once, at the top of the notebook. Everything below depends on this cell having run.

INPUT → OUTPUT

IN	OUT
nothing	all libraries importable · `GROQ_API_KEY` available via `os.environ`

FIRST: GET YOUR GROQ KEY AND STORE IT AS A COLAB SECRET

Do this before running any code. You need a free Groq API key, and the safe place to keep it is Colab's 🔑 **Secrets** panel — *not* typed into a code cell (a key typed in a cell gets pushed to GitHub and disabled). Two steps:

Part A — Create your free Groq key (no credit card):

1. Go to https://console.groq.com and sign up for a free account.
2. In the left sidebar, click **API Keys**.
3. Click **Create API Key**, give it a name (e.g. `ISI Recommender`), and create it.
4. **Copy the key immediately** — it starts with `gsk_` and is shown *only once*. If you lose it, just create another.

Part B — Add it as a Colab Secret named `GROQ_API_KEY`:

1. In your Colab notebook, click the 🔑 key icon in the far-left sidebar to open **Secrets**.
2. Click **+ Add new secret**.
3. In **Name**, type `GROQ_API_KEY` exactly (all caps, with underscores — the code looks for this exact name).
4. In **Value**, paste your `gsk_…` key.
5. Turn the **Notebook access** toggle (left of the row) **ON** — without this, your notebook cannot read the secret.

The `GROQ_API_KEY` row in Colab's 🔑 Secrets panel. **Note this crop is only half-done** — to finish, you must **paste your `gsk_…` value** into the empty Value box *and* switch the **Notebook access toggle (far left) ON**. It only works once *both* are set.

STARTER CODE

```
# run once, at the very top
!pip install -q sentence-transformers groq

import pandas as pd, numpy as np, re, os
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt

# load the Groq key from your Colab Secret into the environment
from google.colab import userdata
os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")
print("setup ok ·", os.environ["GROQ_API_KEY"][:4])   # should print: setup ok · gs
```

Then confirm the key actually works with one tiny Groq call:

```
from groq import Groq
client = Groq()   # automatically reads GROQ_API_KEY from the environment
resp = client.chat.completions.create(
    messages=[{"role": "user", "content": "Say hello in one short line."}],
    model="llama-3.3-70b-versatile",   # verify current models at console.groq.com/
)
print(resp.choices[0].message.content)
```

Two mistakes that cause `GroqError: api_key must be set`:

- **Wrong name.** The secret must be named `GROQ_API_KEY` — not `GROQ` or anything else. The same name appears in `userdata.get("GROQ_API_KEY")`.
- **Secrets are not automatic.** A Colab Secret does *not* appear in `os.environ` on its own — you must pull it with `userdata.get(...)` as shown. `os.environ.get("GROQ_API_KEY")` alone returns `None`.

Quick fallback if Secrets give you trouble: you can instead type `os.environ["GROQ_API_KEY"] = "gsk_...your key..."` directly — but then you **must** delete the key before pushing to GitHub, every time. Secrets avoid that.

📄 **For your report:** note your build environment — Google Colab, Python version, and the libraries you used (with versions: `pd.__version__` etc.). Feeds the report's *Setup / Tools* section.

✅ **Done when:** the setup cell prints `gsk_`, and the test call prints a one-line reply from the model with no red errors.

MODULE 1 **Load data** Sprint 1 · Mtg 2

Purpose. Read the course catalogue into a DataFrame, and define the role→skills table. Do *nothing* else here — no cleaning yet. Just get the raw material into memory.

INPUT → OUTPUT

IN	OUT
a course CSV file (uploaded to Colab) · the role list you decide on	`courses_raw` (DataFrame) · `role_skills` (dict)

THE COURSE CSV — WHAT IT MUST CONTAIN

You're using one fixed dataset: **Coursera Courses & Skills 2024** (Kaggle), the file `coursera_course_dataset_v3.csv` — **623 rows**. Two columns carry the whole pipeline:

COLUMN YOU NEED	REAL NAME IN V3	WHAT IT HOLDS	USED BY
title	`Title`	the course name	part of the search text + shown in recommendations
skills	`Skills`	comma-separated skills the course teaches	the search text (M4/M5) *and* the relevance check (M6)

Step 1 — rename the v3 columns to the names the rest of the notebook expects (the real names are capitalised, so this rename matters):

```
# map the v3 dataset's column names → the names this project expects
courses_raw = courses_raw.rename(columns={
    "Title":              "title",
    "Skills":             "skills",
    "course_description": "description",   # optional (see note)
    "Organization":       "provider",       # optional
    "course_url":         "url",            # optional
    "Difficulty":         "level",          # optional
})
```

Important — we do NOT use the description. v3 has a `course_description` column, but only ~65% of rows have one and some are off-topic. So the search text is built from `title + skills` (both clean and fully populated), *not* from the description. (Details in M2.) `Skills` is the column that matters — it feeds both the search and the scoring.

First, get the file into Colab. The CSV does not appear by itself — upload it using the 📁 **Files panel** on the left of Colab (drag the CSV in), then read it with `pd.read_csv("coursera_course_dataset_v3.csv")`. The file disappears when the Colab session resets, so re-upload if needed (or load it from your shared Google Drive / GitHub repo).

STARTER CODE

```
# 1a. the course catalogue (the v3 file from the Kaggle dataset)
courses_raw = pd.read_csv("coursera_course_dataset_v3.csv")
print(courses_raw.shape)
print(courses_raw.columns.tolist())   # LOOK at the real column names
courses_raw.head()

# 1b. role -> required skills. Hand-made is completely fine to start.
role_skills = {
    "Data Scientist": ["python","statistics","machine learning","sql","data visuali
    "Data Analyst":   ["excel","sql","statistics","data visualization","python"],
    "ML Engineer":    ["python","machine learning","deep learning","sql","cloud"],
    # ... add ~8-10 roles total
}
```

Week-1 reality check: the v3 column names are capitalised (`Title`, `Skills`) — that's exactly why the Step-1 rename matters. Always `print(courses_raw.columns.tolist())` and confirm the names before relying on them — never assume. Report the actual columns at Meeting 2.

📄 **For your report:** record which dataset you chose and *why*, its source URL, how many rows it has, and its real column names. List the roles in your `role_skills` table. Feeds the *Data* chapter.

✅ **Done when:** `courses_raw.head()` shows real rows, and `role_skills["Data Scientist"]` returns a list of skills.

MODULE 2 Clean & prepare

🗓 Sprint 2 · Mtg 3

Purpose. Turn the messy raw catalogue into one tidy table with the exact columns the rest of the notebook expects. The most important output is the `text` column — the single block of words each recommender will match against.

🔁 **Before you start M2:** this module assumes your columns are already called `title` and `skills` — you renamed them in **M1**. If you skipped that rename, the very first line `courses["title"]` will crash with `KeyError: 'title'`. Fix it by going back to M1's rename step — don't patch it here.

WHAT M2 DOES — FIVE SMALL STEPS

1. **Rename** the columns to `title / skills` *(done in M1 — see the callout above)*.
2. **Drop junk rows** — courses missing a title or skills, and duplicate titles.
3. **Renumber** `course_id` to the clean row position (0, 1, 2, …).
4. **Build** `text` — glue `title + skills` into one cleaned, lowercase string. *This is the field the recommenders search.* (We use skills, not the description — see M1's note on why.)
5. **Make** `skills` **a list** — split the skills cell into clean lowercase tags.

INPUT → OUTPUT

IN	OUT: `COURSES` — COLUMNS GUARANTEED
`courses_raw`	`course_id` (int) · `title` (str) · `skills` (list[str]) · `text` (str, cleaned = title + skills)

STARTER CODE

```
courses = courses_raw.copy()

# keep only rows that actually have a title + skills
courses = courses.dropna(subset=["title","skills"]).drop_duplicates("title").reset_
courses["course_id"] = courses.index

def clean(s):
    s = re.sub(r"[^a-z0-9 ]", " ", str(s).lower())
    return re.sub(r"\s+", " ", s).strip()

# the words each recommender will match on = title + skills (NOT description)
courses["text"] = (courses["title"] + " " + courses["skills"]).apply(clean)

# NOW turn skills into a clean lowercase list (after text is built from it)
courses["skills"] = courses["skills"].fillna("").apply(
    lambda s: [x.strip().lower() for x in str(s).split(",") if x.strip()])
```

What `clean()` does, in plain words (the two lines look scary but are simple):
• `re.sub(r"[^a-z0-9 ]", " ", str(s).lower())` — first lower-case everything, then **replace anything that is NOT a letter, number, or space with a space** (so it strips punctuation like `/ , & ( ) :`).
• `re.sub(r"\s+", " ", s).strip()` — **squeeze any run of spaces down to a single space**, and trim the ends.
Result: `"SQL for Data-Analysis (2024)!"` → `"sql for data analysis 2024"`. A tidy lowercase string the recommenders can match on.

⚠️ **The skills split assumes COMMAS.** The line `str(s).split(",")` only works if your dataset separates tags with commas. If it uses `|` or `;` (some do), change the `","` to match — otherwise the whole cell becomes *one giant wrong "skill"* and precision@k (M6) breaks silently. Check with `print(courses["skills"].iloc[0])` — you should see a clean list like `['sql']`, not `['sql | python | regression']`.

WORKED EXAMPLE — ONE REAL COURSE ROW THROUGH M2

A real v3 course — *"Learn SQL Basics for Data Science"* — going from raw to clean. This is exactly what your code produces:

	BEFORE — IN `COURSES_RAW` (AFTER THE M1 RENAME)	AFTER — IN `COURSES`
`course_id`	(its original row index)	a clean row number 0…N−1
`title`	Learn SQL Basics for Data Science	Learn SQL Basics for Data Science *(unchanged)*
`skills`	`"Databases, SQL, Data Management, Data Analysis, Big Data, …"` (a string)	`["databases", "sql", "data management", "data analysis", "big data", …]` (a list)
`text`	*does not exist yet*	`"learn sql basics for data science databases sql data management data analysis big data data model exploratory data analysis …"`

How the `text` value was made: glue `title` + a space + the `skills` string, then run `clean()` (lowercase, strip the commas and punctuation). Note `title` and `skills` aren't themselves changed — they're the raw material for `text`. When a learner's gap contains `"sql"`, this course scores high **because "sql" is in its `text`** — that is the entire matching mechanism. *(We build `text` from skills, not description, because v3's description is sparse/noisy — see M1.)*

Note on the v3 skills: they're comma-separated *phrases* (e.g. `"Statistics for Data Science"`, `"Probability & Statistics"`), which the split above turns into clean lowercase tags. That phrasing is exactly why M6's relevance check uses **whole-word matching**, not exact equality.

For your report: document every cleaning decision — rows dropped, how you built the `text` and `skills` columns, and how many courses survived (before → after counts). Reviewers reward honest data hygiene. Feeds the *Data* chapter.

Done when: `courses.columns` contains `course_id, title, skills, text`, and `courses["text"].iloc[0]` is clean lowercase words.

MODULE 3 User input + skill gap

Sprint 2 · Mtg 3

Purpose. Take the two things the user provides (skills they have + the role they want), look up what that role needs, and compute what is *missing*. The gap — not the skills they have — is what we search on.

INPUT → OUTPUT

IN	OUT
`current_skills` (list) · `target_role` (str) · `role_skills`	`skill_gap` (list) · `query_text` (str)

STARTER CODE

```
# the only two things the user provides
current_skills = ["python"]
target_role    = "Data Scientist"

# Rule 2: the role must exist — fail loudly if not
assert target_role in role_skills, f"Pick a role from {list(role_skills)}"

have      = {s.lower() for s in current_skills}
# the skills still missing, kept in the role table's order (foundational first)
skill_gap = [s for s in role_skills[target_role] if s.lower() not in have]
query_text = " ".join(skill_gap)         # the search query, as one string

print("Skill gap:", skill_gap)
```

Reading that code, line by line (a few bits are new if you only know basic Python):

- `assert target_role in role_skills, "..."` — **stop immediately with a clear message** if the role isn't in the table. This is "fail loudly" — far better than the code limping on and giving empty results (Rule 2).
- `have = {s.lower() for s in current_skills}` — this `{ }` builds a **set** (a bag of unique items) of the user's skills, lower-cased. A set is used because checking "is X in here?" is fast and case is normalised.
- `skill_gap = [s for s in role_skills[target_role] if s.lower() not in have]` — read it as: "**go through each skill the role needs; keep it only if the user doesn't already have it.**" What's left is the gap.
- `" ".join(skill_gap)` — glue the gap skills into one space-separated string, because the recommenders search on a single block of text.

RIYA'S WORKED EXAMPLE

NEEDED (DATA SCIENTIST)	HAS	→ SKILL_GAP
python, statistics, machine learning, sql, data visualization, deep learning	python	**statistics, machine learning, sql, data visualization, deep learning**

```
query_text = "statistics machine learning sql data visualization deep learning"
```

📄 **For your report:** explain the skill-gap idea in your own words and walk through Riya's example (needed − has = gap). State which roles your system supports. Feeds the *Methodology* chapter.

✅ **Done when:** for Riya, `skill_gap` has 5 items and `query_text` is the 5 skills joined by spaces.

MODULE 4 TF-IDF recommender (the baseline)

🗓 Sprint 1→2 · Mtg 2–3

Purpose. The simple, honest baseline. Turn every course's `text` into a word-frequency vector, turn the `query_text` into the same kind of vector, and rank courses by how similar they are. No machine learning training — just counting and weighting words.

NEW CONCEPT — TF-IDF & COSINE SIMILARITY (READ THIS FIRST)

TF-IDF turns a piece of text into a list of numbers (a "vector") by counting its words — but cleverly: a word counts for *more* if it is rare across all courses (distinctive), and *less* if it shows up everywhere (like "course" or "learn").

Tiny example — three one-line courses:

COURSE	ITS WORDS
C1	python · machine · learning
C2	python · data · analysis
C3	yoga · meditation

`python` appears in 2 of 3 courses → common → *low* weight. `yoga` appears in only 1 → rare → *high* weight. So TF-IDF gives each course a set of numbers that emphasise its **distinctive** words.

Cosine similarity then scores how alike two of these number-vectors are, from `0` (nothing in common) to `1` (identical). We turn the user's skill gap into the same kind of vector and rank every course by its cosine similarity to it.

> For the query `"machine learning"`: **C1** (python machine learning) shares both words → high score (≈0.8) → recommended. **C3** (yoga meditation) shares nothing → score 0 → not recommended. That ranking is exactly what this module outputs.

INPUT → OUTPUT

IN	OUT: `TFIDF_RECS`
`courses.text` · `query_text`	top-K DataFrame: `course_id, title, score, rank`

STARTER CODE

```
vec    = TfidfVectorizer(stop_words="english")
matrix = vec.fit_transform(courses["text"])     # every course → a vector
q_vec  = vec.transform([query_text])              # the gap → same vector space

scores = cosine_similarity(q_vec, matrix).flatten()  # one score per course

K   = 10
top = scores.argsort()[::-1][:K]                  # indexes of the K best
tfidf_recs = courses.loc[top, ["course_id","title"]].copy()
tfidf_recs["score"] = scores[top]
tfidf_recs["rank"]  = range(1, len(tfidf_recs)+1)
tfidf_recs
```

The one cryptic line, decoded: `scores.argsort()[::-1][:K]`

- `scores.argsort()` → gives the course *positions* sorted from **lowest score to highest** (worst → best).
- `[::-1]` → **reverses** it, so now it's best → worst.
- `[:K]` → takes the **first K**, i.e. the top-K courses.

So the whole line means "**give me the row positions of the K highest-scoring courses**." *(The same trick appears in M5 — you'll recognise it.)*

Imports: `TfidfVectorizer` and `cosine_similarity` are imported once in the **M0 setup cell** — you don't re-import them here. If you get `NameError: TfidfVectorizer is not defined`, you simply haven't run the M0 cell yet.

Common error: calling `vec.fit_transform()` again on the query. Use `vec.transform()` for the query — the vectorizer was already *fitted* on the courses, and the query must live in that same vocabulary.

📄 **For your report:** explain TF-IDF + cosine similarity in plain English, and paste a sample top-10 table for one user (e.g. Riya). Feeds *Methodology (baseline)*.

✅ **Done when:** `tfidf_recs` has exactly K rows, ranks 1…K, and scores descending.

MODULE 5 — Embedding recommender (the smart version)

🗓 Sprint 2 · Mtg 3

Purpose. The same idea as Module 4 — rank courses by similarity to the gap — but using a pretrained model that understands *meaning*, not just exact words. So "ML" and "machine learning" land close together, where TF-IDF sees two unrelated words.

NEW CONCEPT — EMBEDDINGS (READ THIS FIRST)

Like TF-IDF, **embeddings** turn text into numbers — but instead of *counting words*, a pretrained AI model *reads* the text and produces numbers that capture its **meaning**. Texts that mean similar things get similar numbers, even when they use completely different words.

Why that matters — the same query, `"machine learning"`, against two courses:

COURSE	TF-IDF (EXACT WORDS)	EMBEDDINGS (MEANING)
"machine learning fundamentals"	✅ high — shares the words	✅ high
"deep neural networks & AI"	❌ ≈0 — no shared words	✅ high — related meaning

TF-IDF *misses* the second course because the words don't overlap; embeddings *catch* it because the model knows the topics are related. **Whether that actually gives better recommendations is exactly what you measure in Module 6.**

Notice: the ranking logic (cosine similarity → argsort → top-K) is *identical* to Module 4. **The only thing that changes is how text becomes numbers.** Make this point in your report — it is the cleanest way to explain the comparison.

INPUT → OUTPUT

IN	OUT: `EMBED_RECS`
`courses.text` · `query_text`	same shape as `tfidf_recs` : `course_id, title, score, rank`

STARTER CODE

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")     # small, free, runs in Colab

course_emb = model.encode(courses["text"].tolist(), show_progress_bar=True)
q_emb      = model.encode([query_text])

scores = cosine_similarity(q_emb, course_emb).flatten()   # SAME ranking as M4

K   = 10
top = scores.argsort()[::-1][:K]
embed_recs = courses.loc[top, ["course_id","title"]].copy()
embed_recs["score"] = scores[top]
embed_recs["rank"]  = range(1, len(embed_recs)+1)
embed_recs
```

First run is slow (it downloads the model, ~90 MB, and encodes every course). That is normal. Run it once; Colab keeps it in memory after that. Keep `K` the same value as Module 4 so the comparison is fair.

📄 **For your report:** explain how embeddings differ from TF-IDF (meaning vs exact words), and show the *same* user's top-10 side by side with Module 4 so the contrast is visible. Feeds *Methodology (embeddings)*.

✅ **Done when:** `embed_recs` has the same columns and K rows as `tfidf_recs`, and the two lists are *different* (if identical, something is wrong).

MODULE 6 **Evaluate & compare ★** 🗓 Sprint 3 · Mtg 4

Purpose. This is the graded heart of the project. Measure how good each recommender is using **precision@k** — of the top-K courses it recommended, how many were genuinely relevant (per the locked `is_relevant()` rule) — then compare, chart, and declare a winner.

NEW CONCEPT — WHAT IS PRECISION@K? (READ THIS FIRST)

You have probably seen *precision* in theory. **precision@k** is the version used for ranked lists like recommendations. Plain definition:

precision@k = out of the top *k* courses the recommender suggested, the fraction that were actually relevant.

Worked example. Say the recommender returns its **top 5** courses for Riya (so `k = 5`). We check each one: does it teach a skill Riya is missing?

RANK	RECOMMENDED COURSE	TEACHES A MISSING SKILL?
1	SQL for Data Analysis	✅ yes (sql)
2	Intro to Machine Learning	✅ yes (machine learning)
3	Yoga for Beginners	❌ no
4	Deep Learning Specialization	✅ yes (deep learning)
5	Cooking 101	❌ no

3 of the 5 were relevant, so:

```
precision@5 = 3 / 5 = 0.6
```

Higher is better. A precision@5 of 1.0 means every one of the top 5 was useful; 0.0 means none were. You compute this same number for *both* recommenders and compare — the higher one wins.

> **The one question this raises:** who decides the ✅ / ❌ in that table? **You do — with one rule**, fixed in §03: a course counts as relevant if one of the gap skills appears (as a whole word) in its skills. That rule is `is_relevant()`, and it is the yardstick behind every score here.

INPUT → OUTPUT

IN	OUT
`tfidf_recs` · `embed_recs` · `courses` · `skill_gap`	`eval_table` (DataFrame) · a bar chart · `winner` (str)

STARTER CODE

```
def is_relevant(course_skills, skill_gap):
    skills_text = " ".join(course_skills).lower()
    return any(re.search(r"\b" + re.escape(g) + r"\b", skills_text) for g in skill_

def precision_at_k(recs, courses, skill_gap, k=10):
    hits = 0
    for cid in recs.head(k)["course_id"]:
        cskills = courses.loc[courses["course_id"]==cid, "skills"].iloc[0]
        hits += is_relevant(cskills, skill_gap)
    return hits / k

p_tfidf = precision_at_k(tfidf_recs, courses, skill_gap, k=10)
p_embed = precision_at_k(embed_recs, courses, skill_gap, k=10)

eval_table = pd.DataFrame({"method":["TF-IDF","Embedding"],
                           "precision@10":[p_tfidf, p_embed]})
winner = "Embedding" if p_embed >= p_tfidf else "TF-IDF"

eval_table.plot.bar(x="method", y="precision@10", legend=False)
plt.title("Which recommender is better?"); plt.ylabel("precision@10"); plt.show()
```

This is the most important code in the project — so here it is line by line:

- `re.search(r"\b" + g + r"\b", skills_text)` — checks whether the gap skill `g` appears as a **whole word** in the course's skills text; `any(... for g in skill_gap)` asks "**does at least one gap skill show up?**" → that's what makes a course "relevant". (Whole-word, so `statistics` matches `"statistics for data science"` but `sql` won't falsely match `"mysql"`.)
- `hits += is_relevant(...)` — `is_relevant` returns `True` / `False`, and in Python `True` **counts as 1,** `False` **as 0**. So this adds 1 for every relevant course. After the loop, `hits` = how many of the top-k were relevant.
- `courses.loc[courses["course_id"]==cid, "skills"].iloc[0]` — looks dense, reads simply: "**find the row whose** `course_id` **is** `cid`**, and take its** `skills` **list.**" The `.iloc[0]` just grabs that single value out of the one-row result.
- `hits / k` — relevant-count ÷ how-many-we-checked = **precision@k**. Exactly the 3/5 = 0.6 from the example above.

Imports: `pd` (pandas) and `plt` (matplotlib) come from the **M0 setup cell**. `is_relevant()` is the *same* rule locked in §03 — define it once and reuse it; don't write a second version.

Keep `k` **here equal to the** `K` **you chose in M4/M5.** If M4 returns only 5 courses but you call `precision_at_k(..., k=10)`, it divides by 10 while checking 5 — the score comes out wrongly low and the comparison is unfair.

Go further for more marks: compute precision@k for several values of k (3, 5, 10), test several different roles (not just Data Scientist), and average. **This matters here:** on this dataset a single role can score 1.0 at k=10 for *both* methods (a tie) — so testing several roles and k values is what actually reveals the difference between TF-IDF and embeddings. *This is where your ambition belongs.*

📄 **For your report — this is your *Results* chapter:** paste the `eval_table`, the bar chart, and 2–3 sentences interpreting *which method won and why*. Critically discuss the limits of your `is_relevant()` rule. This section carries the most marks — invest here.

✅ **Done when:** `eval_table` shows two precision scores, the bar chart renders, and `winner` holds one of the two method names.

MODULE 7 Pick best + LLM roadmap (Groq)

🗓 Sprint 4 · Mtg 5

Purpose. Take the winning recommender's top courses and ask a free LLM to turn the bare list into an encouraging, human, month-by-month roadmap. This is the "flourish" — one API call, on top of a study that already stands on its own.

INPUT → OUTPUT

IN	OUT
`winner` · `tfidf_recs` / `embed_recs` · `current_skills` · `target_role`	`roadmap_text` (str)

STARTER CODE

```
from groq import Groq
client = Groq(api_key=os.environ["GROQ_API_KEY"])

best_recs   = embed_recs if winner=="Embedding" else tfidf_recs
course_list = "\n".join(f"- {t}" for t in best_recs["title"].head(8))

prompt = f"""You are helping a learner plan their upskilling.

Current skills: {', '.join(current_skills)}
Target role: {target_role}
Skills they still need (the gap): {', '.join(skill_gap)}

Recommended courses — use ONLY these, do not invent or add any:
{course_list}

Write a month-by-month learning roadmap that:
- uses ONLY the courses listed above,
- orders them sensibly: foundational skills (e.g. SQL, statistics) before
  advanced ones (e.g. machine learning, deep learning),
- gives one short sentence per course on why it sits at that point,
- is realistic for a part-time learner over ~6 months,
- ends with one line stating this is a SUGGESTED order, not a verified
  prerequisite plan.

Keep the tone encouraging and concise."""

resp = client.chat.completions.create(
    model="llama-3.3-70b-versatile",          # verify current models at console.gr
    messages=[{"role":"user","content":prompt}])

roadmap_text = resp.choices[0].message.content
print(roadmap_text)
```

Auth error / `KeyError: 'GROQ_API_KEY'` **?** Your key isn't loaded. It's set once in the **M0 setup cell** (`os.environ["GROQ_API_KEY"] = "gsk_..."`) — re-run M0, then this cell. Never paste the key directly here, and never commit it to GitHub.

Model names change. If you get a "model not found" error, the model name was retired — check the current free models at `console.groq.com/docs/models` and swap it in. Do not guess.

Limitation to own in your report: the roadmap's *ordering* is the LLM's best guess from general knowledge — the system does **not** model real course prerequisites (no dataset carries that). Present it as a *suggested* order, not a verified one. Note in your report that true prerequisite sequencing (a prerequisite graph + topological sort) is *future work*. Also: the "use ONLY these courses" wording in the prompt is what stops the LLM inventing courses — keep it.

📄 **For your report:** include one full example roadmap (Riya's), and 2–3 sentences on what the LLM adds and its limits (it can't verify the courses; it just narrates). Feeds the *Conclusion / extensions* section (Student 5).

✅ **Done when:** `roadmap_text` is a multi-paragraph roadmap that names courses from `best_recs`. If the call fails, the project still passes — this step is optional polish.

MODULE 8 **Optional UI — only if everything else is done** 🗓 Sprint 4–5 · optional

Purpose. Wrap Modules 3→7 behind simple input boxes so a non-coder can use it. **Do not start this until the notebook is complete and the report is drafted.** It earns no extra marks on its own — it is a demo nicety.

The Handbook calls this the *optional demo*. Two ways to build it: **`ipywidgets`** (simplest — stays inside the notebook, shown below) or a **`streamlit`** app (separate file, more work). Either is fine; both are optional.

SIMPLEST VERSION — INSIDE THE NOTEBOOK, NO EXTRA TOOLS

```
import ipywidgets as widgets
skills = widgets.Text(description="Your skills:")            # comma-separated
role   = widgets.Dropdown(options=list(role_skills), description="Target role:")
go     = widgets.Button(description="Get my roadmap")
display(skills, role, go)
# on click: read the boxes, then re-run Modules 3 → 7 with those values
```

Remember Rule 2: the role is a `Dropdown` (fixed choices), never a free text box — otherwise an unknown role silently returns nothing.

📄 **For your report:** if you built it, add a screenshot and a line on how it works. If you didn't, list it as *future work* in the Conclusion — that is a perfectly good outcome. (Student 5)

✅ **Done when:** (only if attempted) typing skills + picking a role + clicking the button prints recommendations and the roadmap.

06

Plain-English glossary

TERM	IN ONE SENTENCE
TF-IDF	A way to turn text into numbers by counting words, giving rare-but-meaningful words more weight than common ones.
Embedding	A way to turn text into numbers using a pretrained model, so that pieces of text with similar *meaning* get similar numbers.
Cosine similarity	A score from 0 to 1 measuring how close two vectors point in the same direction — i.e. how similar two pieces of text are.
Skill gap	The skills a target role needs minus the skills the person already has = what they still need to learn.
precision@k	Out of the top *k* courses recommended, the fraction that were genuinely relevant. Higher = better recommender.
Ground truth	The "correct answer" you measure against. Here: our `is_relevant()` rule defines it.
Baseline	The simplest reasonable method (TF-IDF), used as a yardstick to judge whether the fancier method (embeddings) is actually worth it.